

Complete User Guide

All features, credentials & configuration reference

Bernstein Feedback — Complete User Guide

End-to-end guide to every feature shipped in this codebase, organised the same way the product document is. Each section starts with **what the doc promises** and ends with **how to use it**. The final section is a complete credentials and configuration reference for every key, env var, and account you need to run the system.

Table of contents

Part 1 — For end users (what they see in the host product)

1. The floating bubble
2. Submitting feedback, bugs, and feature requests
3. Loop-close badge & resolution emails
4. Proactive rage-click prompt

Part 2 — For developers (install + integrate)

5. Install the widget 6. Wire up PostHog session replay 7. PostHog automatic error tickets (webhook)

Part 3 — For project admins

8. Admin dashboard tour 9. Triage workflow 10. AI ticket clustering & cluster siblings 11. Auto-resolvable fixes — one-click diff approval 12. Feedback loop health card 13. Plans, billing, and limits

Part 4 — For AI agents

14. Agent API — backlog, notes, close

Part 5 — Reference

15. Feature matrix vs the product doc 16. Credentials & configuration reference ← **start here when setting up**

Part 1 — For end users

1. The floating bubble

Doc: "Once installed, it appears as a floating chat bubble in the bottom-right corner of the host application."

A round button sits in the bottom-right corner of the host app once the widget is mounted. It's always there, doesn't block content, and works without any user account.

2. Submitting feedback, bugs, and feature requests

Doc: "The component currently supports three submission types: a bug report, a feature request, or general feedback."

Click the bubble → modal opens with three tabs:

- **Feedback** — anything (compliments, confusion, ideas)
- **Bug report** — what's broken + (optionally) the element that's broken
- **Feature request** — what's missing

What gets captured automatically:

- **Page and view** — URL, route, page name
- **Navigation history** — every route the user passed through to get here
- **Component pinpointing** — click the target icon, then click any element to attach its CSS selector + bounding box + text
- **Screenshot** — auto-captured (the dialog itself is hidden during capture); user can retake or remove
- **Debug info** — browser, OS, viewport, console errors, failed network requests (no request bodies — privacy)

User input:

- Free-text title (required) and description (optional)
- Optional category, severity, impact (only on bug tab)
- Optional email (auto-attached if `userEmail` is set in widget config)

Privacy:

- No keystrokes, no form data, no request bodies are ever captured
- Auto-redaction strips emails, phone numbers, API-key-shaped strings from user text
- Per-submission consent toggles let the user opt out of any context category

3. Loop-close badge & resolution emails

Doc: "When a ticket is resolved, the bubble icon displays a numeric badge so the user knows something has changed."

Two channels close the loop:

In-app badge — when an admin marks the user's ticket resolved, the bubble shows a numeric badge. Clicking the bubble opens the notifications panel where the user sees: "Your feedback 'X' has been resolved."

Email — if SMTP is configured on the server AND the submission included an email, the user gets a branded resolve email with the developer's resolution note.

Cluster fan-out — if the resolved ticket was part of a cluster (AI grouped 5 reporters under one issue), **every** reporter gets the badge and the email. Each unique recipient gets exactly one of each (deduped by `email_queue.dedupe_key` and trigger guard).

4. Proactive rage-click prompt

Doc: "PostHog can trigger in-app prompts after errors, rage clicks, or abandoned flows, increasing feedback volume from users who would otherwise leave silently."

When the user clicks the same element 4+ times in 1.5 seconds — the classic frustration signal — a small card appears bottom-left:

"Something not working? We noticed a few clicks on `<button#submit>` . If it's broken, tell us and we'll take a look."

- **Report it** opens the bug dialog with title and description prefilled.
- **Not now** dismisses it; it won't reappear in the same session.
- Auto-dismisses after 15 seconds if ignored.
- Frequency cap: at most one prompt per session, per trigger type.

Distinct from the bubble (bottom-right) so users see two clearly separate affordances.

Part 2 — For developers

5. Install the widget

Doc: "The developer adds the Bernstein Feedback node component to their application and passes the project ID into the component configuration."

```
npm install @bernstein/feedback
```

```

import { FeedbackProvider, FeedbackButton } from '@bernstein/feedback';
import { httpAdapter } from '@bernstein/feedback/adapters';
import '@bernstein/feedback/styles.css';

export default function App() {
  return (
    <FeedbackProvider
      config={{
        projectId: import.meta.env.VITE_BERNSTEIN_PROJECT_ID,
        adapter: httpAdapter({
          endpoint: import.meta.env.VITE_BERNSTEIN_SERVER + '/api/feedback',
        }),
        userId: currentUser?.id,
        userEmail: currentUser?.email,
        appVersion: '1.4.2',
        env: 'production',

        // OPTIONAL – see sections 4, 6
        proactiveTriggers: { rageClick: true },
      }}
    >
      <YourApp />
      <FeedbackButton />
    </FeedbackProvider>
  );
}

```

Where to get values:

- `projectId` — admin dashboard → Settings → Project ID (copy)
- Server endpoint — your deployed Bernstein server (or `https://api.bernstein.example.com`)

6. Wire up PostHog session replay

Doc: "Every ticket with an attached session ID shows a direct deep link into PostHog. The developer clicks it and lands on the exact session replay for that user at that moment."

If your app already runs PostHog:

```

import posthog from 'posthog-js';
import { posthogSessionProvider } from '@bernstein/feedback';

<FeedbackProvider
  config={{
    projectId: '...',
    adapter: httpAdapter({ endpoint: '...' }),
    sessionProvider: posthogSessionProvider(posthog),
  }}
>

```

Effect:

- Every ticket gains `session_id`, `session_replay_url`, `user_properties` automatically
- Admin detail page shows a **View session replay** deep-link button

- Admin detail page shows a **User properties** card (plan, signup date, anything you `posthog.identify()` - ed)

Other providers (LogRocket, FullStory) implement the same `SessionProvider` interface — see [packages/feedback/src/sessionProviders/](#).

Plan gate: `features.posthog` (paid plan).

7. PostHog automatic error tickets (webhook)

Doc: "PostHog error tracking can be configured to automatically open a Bernstein ticket when a JS exception crosses a threshold, with no user action required."

Have PostHog POST to a webhook on the Bernstein server when an exception fires. A bug_report row appears in your dashboard automatically — replay URL, user identity, exception stack all attached.

Setup:

1. Get your project's `api_key` from the admin dashboard → Settings → Project API key (or `SELECT api_key FROM projects WHERE id='YOUR_PID'`).
2. In PostHog → Data Pipelines → new webhook destination:
 - **Trigger:** event = `$exception`
 - **URL:** `https://your-bernstein-server/api/v1/integrations/posthog/YOUR_PROJECT_ID/error`
 - **Method:** POST
 - **Headers:** X-API-Key: <your api_key>
 - **Body:** PostHog's default event payload (we read `properties.$exception_message` , `$exception_type` , `$session_id` , `$session_recording_url` , `$current_url` , `email`)

Test locally:

```
curl -X POST \
-H "X-API-Key: YOUR_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "distinct_id": "user-123",
  "properties": {
    "$exception_type": "TypeError",
    "$exception_message": "Cannot read property cart of undefined",
    "$current_url": "https://app.example.com/checkout",
    "$session_id": "sess-abc",
    "$session_recording_url": "https://app.posthog.com/replay/sess-abc",
    "email": "user@example.com"
  }
}' \
https://your-bernstein-server/api/v1/integrations/posthog/YOUR_PROJECT_ID/error
```

Dedup: same `distinct_id + $exception_type + date` collapses to one ticket. Repeat calls return `{ deduplicated: true }` and don't consume quota.

Plan gate: `features.posthog` (paid plan). Returns 429 if the project hit its monthly limit.

Part 3 — For project admins

8. Admin dashboard tour

Doc: "The developer creates a project in the dashboard. Each project receives a unique project ID."

After login the dashboard shows:

- **Header** — project switcher dropdown, notification bell (cross-project), user menu
- **Feedback list** — every ticket from every accessible project. Filters: type / status / priority / severity. Clusters collapse to one row with a `xN` badge.
- **Feedback detail** — title, description, screenshots, navigation path, console + network errors, user actions, session replay link, cluster siblings, agent notes, proposed fix panel, triage controls, status controls, raw JSON
- **Stats** — total / by-type / by-severity bar charts, monthly usage gauge with upgrade CTA
- **Settings** — project metadata, project API key, BYOK AI key, plan upgrade button, members

9. Triage workflow

Doc: "All incoming tickets appear in the developer's project management dashboard. The developer can review, label, and prioritise them."

On the detail page:

- **Status** — `open` / `in_progress` / `resolved` / `closed` . Resolving fires loop-close notifications + emails.
- **Resolve note** — optional text appended to the resolve email and notification.
- **Priority** — `low` / `medium` / `high` / `urgent` . Click again to clear.
- **Labels** — free-form tags (`backend` , `duplicate` , `ux-bug`). Spaces become hyphens.

10. AI ticket clustering & cluster siblings

Doc: "When a ticket arrives at the Bernstein server, it is passed to an embedding model... Tickets with high similarity, same page, same component, similar description, are grouped into a cluster."

A background worker (`cluster worker`) embeds new feedback via OpenAI's `text-embedding-3-small` and assigns it to the nearest existing cluster (cosine similarity ≥ 0.85). New clusters start single-member; subsequent matches bump the count.

What you see:

- **Feedback list** — clustered rows show one row with a `xN` badge instead of N copies
- **Detail page** — "Also reported by N other users" card lists every sibling reporter
- **Resolve fan-out** — resolving any one row notifies every cluster reporter
- **Priority score** — frequency $\times$ recency $\times$ paid-user weight (sortable in the Agent API)

Bring your own OpenAI key (BYOK):

Paid-plan project owners → admin Settings → AI key → paste an OpenAI key. The server encrypts it at rest with pgcrypto. The cluster worker uses your key for that project's embeddings; cost lives on your OpenAI

account, not the platform's.

Plan gate: `features.ai_clustering` (paid plan).

11. Auto-resolvable fixes — one-click diff approval

Doc: "For a small subset of tickets that are narrow and self-contained, such as an incorrect colour, a missing null check, or a label typo, the system flags these as auto-resolvable. The developer sees a one-click fix with a full diff preview."

A SQL classifier (`classify_cluster_auto_resolvable`) flags clusters as narrow-fix candidates when:

- type is `bug_report`
- combined title + description $\leq$ 600 chars
- text matches narrow-fix keywords: `typo` , `misspell` , `wrong text` , `wrong label` , `color` , `colour` , `css` , `margin` , `padding` , `alignment` , `null check` , `undefined` , `nullref` , `nil pointer` , `404 on` , `broken link` , `dead link`

When an AI agent attaches a diff via the Agent API (`POST /clusters/:id/propose-fix`), the admin detail page shows a **Proposed Fix** card:

```
Proposed Fix
Fix typo: Welcom → Welcome
Proposed by claude-code · 2026-04-23 10:14
Confidence 95%
[src/Welcome.tsx]

--- a/src/Welcome.tsx
+++ b/src/Welcome.tsx
@@ -1,3 +1,3 @@
-<h1>Welcom</h1>
+<h1>Welcome</h1>

[Approve & Resolve]
```

Clicking **Approve & Resolve** closes every reporter in the cluster (one fan-out, not N) and records the approval as `resolved_by = admin:<email>` , with `resolution_note = "Auto-fix: <summary> (approved by <admin email>)"` .

The actual code change happens in your repo via the agent (Codex / Claude Code) — Bernstein orchestrates approval and loop close, it doesn't push commits.

12. Feedback loop health card

Doc: "The dashboard includes a feedback loop health card showing three metrics: average time from submission to resolution, percentage of tickets closed within 14 days, and end-user return rate."

On the Stats page, traffic-light card:

Metric	Green	Amber	Red
Avg resolution hours	≤ 48h	≤ 168h (7d)	> 7d
% closed in 14 days	≥ 80%	≥ 50%	< 50%
Return rate	≥ 40%	≥ 15%	< 15%

Computed by the `feedback_loop_health()` SQL function. When return rate drops to amber/red, the doc says it's a signal that users are submitting and not coming back — close a few tickets to send notifications and rebuild engagement.

13. Plans, billing, and limits

Doc: "The free plan is permanent... Paid plans increase the ticket volume, support multiple projects, and unlock advanced features."

Feature	Free	Paid
Projects	1	unlimited
Tickets / month	50	unlimited
AI clustering	✗	✓
PostHog integration + webhook	✗	✓
Agent API	✗	✓
Loop-close emails + notifications	✓	✓
Proactive prompts	✓	✓

At the free limit: the widget enters **read-only mode** — bubble still renders, resolved notifications still deliver, new submissions show *"This project has reached its monthly feedback limit."* and don't create tickets. The owner gets a one-time email at 80% and another at 100%.

Upgrading: admin Settings → Upgrade. If `BILLING_PROVIDER` env is unset, the plan flips immediately (self-hosted). If set to `stripe` (or any value), the route returns 501 until you wire a real payment webhook (the seam is in place; the Stripe handler is not).

Part 4 — For AI agents

14. Agent API — backlog, notes, close

Doc: "Developers on paid plans can connect an AI coding assistant, such as Codex or Claude Code, to the Bernstein Feedback API. The agent reads the prioritised ticket backlog and does the archaeology."

Auth: `X-API-Key: <projects.api_key>` . Plan gate: `features.api_access` (paid).


```
# Top of the prioritised backlog (clusters + unclustered, sorted by priority_score)
GET /api/v1/agent/:projectId/backlog?type=bug_report&limit=20&include_resolved=false

# Full investigation context – every member, console errors, network errors, breadcrumbs
GET /api/v1/agent/:projectId/clusters/:clusterId

# Investigation note (doesn't change status, can leave many)
POST /api/v1/agent/:projectId/feedback/:feedbackId/note
Body: { "note": "...", "author": "claude-code" }

# Proposed fix (renders in admin "Proposed Fix" card)
POST /api/v1/agent/:projectId/clusters/:clusterId/propose-fix
Body: { "summary", "diff", "files", "confidence", "proposed_by" }

# Close the whole cluster – fans out resolve notifications + emails
# to every reporter exactly once (load-bearing trigger guard)
POST /api/v1/agent/:projectId/clusters/:clusterId/close
Body: { "resolution_note", "actor" }
```

Full reference + example agent loop in section 9 of the [test plan](#).

Part 5 — Reference

15. Feature matrix vs the product doc

Doc section	Status	Where it lives
Floating bubble + 3-tab dialog	✓ shipped	<code>packages/feedback/src/components/</code>
Page, navigation, screenshot, debug info capture	✓ shipped	<code>context.tsx</code>
Component-level element targeting	✓ shipped	<code>FeedbackDialog.tsx</code>
Loop-close badge in widget	✓ shipped	<code>useNotifications</code> hook
Loop-close email	✓ shipped	<code>queue_email_on_feedback_resolved</code> trigger + <code>emailWorker</code>
Read-only mode at limit	✓ shipped	server <code>feedback.ts</code> + widget gate
Free / Paid plan gating	✓ shipped	<code>requirePlanFeature</code> middleware
Project creation + API key	✓ shipped	admin dashboard
AI ticket clustering	✓ shipped	<code>clusterWorker.ts</code>
BYOK OpenAI keys	✓ shipped	<code>aiKeys.ts</code>
Cluster siblings panel	✓ shipped	admin detail page
Priority score	✓ shipped	<code>feedback_cluster_priority()</code> SQL
Feedback loop health card	✓ shipped	<code>feedback_loop_health()</code> SQL + Stats page
PostHog session replay (sessionProvider)	✓ shipped	<code>posthog.ts</code>
PostHog automatic error → ticket webhook	✓ shipped	<code>integrations.ts</code>
Proactive rage-click prompt	✓ shipped	<code>useRageClickDetector.ts</code>

Doc section	Status	Where it lives
AI agent API	✓ shipped	agent.ts
Auto-resolvable flagging	✓ shipped	<code>classify_cluster_auto_resolvable()</code> SQL
One-click diff preview UI	✓ shipped	admin <code>FeedbackDetailPage.tsx</code>
Self-hosted server (Postgres)	✓ shipped	self-host via <code>server/init.sql</code>
Supabase deployment (RLS)	✓ shipped	<code>supabase-setup.sql</code>
Cross-project admin notification bell	✓ shipped	<code>useAdminNotifications</code>
Self-serve plan upgrade route	✓ shipped	<code>POST /api/projects/:id/upgrade</code> (Stripe handler stubbed)
Assumption validation	✗ roadmap	not started
Feature voting	✗ roadmap	not started
Public changelog	✗ roadmap	not started
PostHog proactive prompts (other than rage click)	🕒 partial	only <code>rage_click</code> implemented; abandoned-flow / error-burst not yet

16. Credentials & configuration reference

Everything you need to gather to run the system, organised by what you're enabling. Required vs optional is per-feature.

A) Run the server (always required)

Variable	Where to get it	Required?
<code>DATABASE_URL</code> or <code>DB_HOST/DB_PORT/DB_USER/...</code>	Supabase dashboard → Settings → Database → Connection string (transaction pooler), or your own Postgres	✓ yes
<code>JWT_SECRET</code>	Generate: <code>openssl rand -hex 32</code>	✓ yes (otherwise dev fallback is used)
<code>PORT</code>	default <code>3000</code>	optional
<code>ALLOWED_ORIGINS</code>	comma-separated list of admin/widget origins (CORS)	✓ for production
<code>NODE_ENV</code>	<code>production</code> / <code>development</code>	optional

Apply schema:

```
psql $DATABASE_URL -f server/init.sql # fresh self-hosted
# OR for Supabase
psql $DATABASE_URL -f examples/supabase-setup.sql

# Then every additive migration (idempotent):
for f in server/migrations/*.sql; do psql $DATABASE_URL -f "$f"; done
```

B) Email notifications (resolve emails, plan-warning, plan-limit)

Variable	Where to get it	Required?
<code>SMTP_HOST</code>	e.g. <code>smtp.gmail.com</code>	✓ for email
<code>SMTP_PORT</code>	<code>587</code> (STARTTLS) or <code>465</code> (SSL)	✓
<code>SMTP_USER</code>	your sending account	✓
<code>SMTP_PASS</code>	Gmail app password (not login password) — myaccount.google.com → Security → 2-Step → App passwords	✓
<code>SMTP_FROM</code>	<code>"Bernstein Feedback <noreply@example.com>"</code>	optional, defaults to <code>SMTP_USER</code>

If unset, the server still boots — emails accumulate in `email_queue` until SMTP is configured. Verify on boot:

```
[email] SMTP verified. Worker polling every 30000 ms
```

C) AI clustering — global OpenAI key

Variable	Where to get it	Required?
<code>OPENAI_API_KEY</code>	https://platform.openai.com/api-keys → create key	one of (C) or (D) for clustering
<code>CLUSTER_POLL_INTERVAL_MS</code>	default <code>30000</code>	optional
<code>CLUSTER_BATCH_SIZE</code>	default <code>10</code>	optional
<code>CLUSTER_SIMILARITY_THRESHOLD</code>	<code>0.0 – 1.0</code> , default <code>0.85</code> . Higher = stricter grouping	optional
<code>CLUSTER_EMBEDDING_MODEL</code>	default <code>text-embedding-3-small</code>	optional

Without this OR the BYOK secret (D), the cluster worker logs once at boot and stays disabled. Feedback submission still works, just without grouping.

D) AI clustering — per-project BYOK keys

Use this when you want each paid customer to fund their own embedding spend.

Variable	Where to get it	Required?
<code>AI_KEY_ENCRYPTION_SECRET</code>	Generate 32+ random chars: <code>openssl rand -hex 32</code>	✅ for BYOK

Then, project owners (paid plan) enter their OpenAI key in admin → Settings → AI key. The server encrypts with pgcrypto + this secret. **Rotating this secret invalidates every stored BYOK key — owners must re-enter.**

Worker resolution order: per-project BYOK key → global `OPENAI_API_KEY` → skip (warn-once-per-project).

E) PostHog session replay (widget side)

Just install posthog-js in your host app and pass the SDK to the widget — no Bernstein-side credentials.

```
import posthog from 'posthog-js';
import { posthogSessionProvider } from '@bernstein/feedback';

<FeedbackProvider config={{ ..., sessionProvider: posthogSessionProvider(posthog) }}>
```

PostHog's own setup needs `posthog.init(POSTHOG_PROJECT_KEY, { api_host })` — that's a PostHog credential, not a Bernstein one.

F) PostHog automatic error webhook (server-inbound)

What	Where	Required?
Project API key	admin → Settings → Project API key (also: <code>SELECT api_key FROM projects WHERE id='YOUR_PID')</code>)	✓
PostHog webhook destination	PostHog → Data Pipelines → new webhook	✓
Webhook URL	<code>https://your-bernstein-server/api/v1/integrations/posthog/YOUR_PROJECT_ID/error</code>	✓
Header	<code>X-API-Key: <project api_key></code>	✓
Plan flag	<code>features.posthog = true</code> (auto on paid plan)	✓

G) Agent API (Codex / Claude Code / CI)

What	Where	Required?
Project API key	admin → Settings → Project API key	✓
Plan flag	<code>features.api_access = true</code> (auto on paid plan)	✓

The agent uses the same key as the PostHog webhook — it's a per-project key, not per-feature.

H) Widget configuration (host app)

Config field	Source	Required?
<code>projectId</code>	admin → Settings → Project ID	✓
<code>adapter</code>	<code>httpAdapter({ endpoint: 'https://your-bernstein-server/api/feedback' })</code> or supabase adapter	✓
<code>userId</code> , <code>userEmail</code>	your auth system	optional but enables loop-close email + identity
<code>appVersion</code> , <code>buildSha</code> , <code>env</code>	your build pipeline	optional, attached to context
<code>proactiveTriggers</code>	<code>{ rageClick: true }</code> to enable the rage-click card	optional
<code>sessionProvider</code>	<code>posthogSessionProvider(posthog)</code> if using PostHog	optional

I) Self-serve plan upgrade (Stripe etc.)

Variable	Required?
<code>BILLING_PROVIDER</code>	optional. If unset, plan flips immediately on <code>POST /api/projects/:id/upgrade</code> . If set to <code>stripe</code> (or anything truthy), the route returns 501 until you wire a real payment handler.

The Stripe handler itself is **not built** — only the seam. Building it: implement provider checkout init in the `if (provider)` branch in `server/src/routes/projects.ts`, add a `POST /:id/billing/confirm` webhook that verifies the signature and flips the plan.

J) WebSocket realtime notifications

No extra credentials — the WebSocket server attaches to the same HTTP port as the REST API at `/api/notifications/ws`. The widget detects and uses it automatically when running against the Node server (Supabase deployments use Supabase Realtime via `pg_notify`).

Quick-start checklist

If you're setting up a fresh deployment, do these in order:

1. **Provision Postgres** → set `DATABASE_URL` → run `init.sql` (or `supabase-setup.sql`) → run every migration in `server/migrations/` in number order.
2. **Generate `JWT_SECRET`** → set `ALLOWED_ORIGINS` → boot the server → `curl http://localhost:3000/health`.
3. **Sign up the first user** in the admin dashboard (auto-promoted to admin).
4. **Create your first project** → copy the Project ID + Project API key.
5. **Install the widget** in your host app with the Project ID and server URL.
6. **(Optional)** Configure SMTP for email notifications.
7. **(Optional, paid)** Add `OPENAI_API_KEY` for clustering, OR set `AI_KEY_ENCRYPTION_SECRET` and let owners enter their own keys.
8. **(Optional, paid)** Wire PostHog: install `posthog-js` → pass `posthogSessionProvider(posthog)` to the widget → optionally set up the error webhook.
9. **(Optional, paid)** Hand the Project API key to your AI agent (Codex / Claude Code) → point it at `/api/v1/agent/:projectId/backlog`.
10. **(Optional)** Enable the proactive prompt: `proactiveTriggers: { rageClick: true }`.

Where to go from here

- **Full test plan for this branch:** `test-plan-agent-cluster-feedback.md`
- **AI clustering deployment deep-dive:** `ai-clustering-setup.md`
- **Adapters reference:** `adapters.md` (in `packages/feedback/docs/`)
- **Widget config full reference:** `configuration.md` (in `packages/feedback/docs/`)
- **Server architecture notes:** `server/CLAUDE.md`